Go Fish 2.0: An Autonomous Fish-Following Robot for Real-Time Behavioral Tracking

Madison Carney, Alexis Phan, Navya Arora, and Esha Madamalla
Date: 05/05/2026
Yilmaz - Period 7
Stickler - Period 6

Table of Contents

Abstract

Spatial learning in fish is important because it provides a measurable way to study navigation, memory, and behavior in a model organism. Goldfish are known to use landmarks, cues, and associative memory, but many fish-learning experiments still depend on manual observation, which is inconsistent, time-consuming, and difficult to quantify precisely. This project completed an autonomous fish-following robot that uses computer vision and LiDAR to track goldfish movement and convert that movement into real-time motor commands. A Raspberry Pi processed camera input using HSV color detection and YOLO-based object detection tests, read distance measurements from an RPLIDAR A1M8 sensor, and sent movement commands to a VEX V5 Brain motor controller over USB serial communication. The system replaced last year's time-based motor control with velocity-based control, integrated LiDAR for obstacle detection, and created a continuous loop for detecting fish position, computing movement values, applying safety overrides, and updating motor behavior. The completed system demonstrated that fish movement can be detected and translated into real-time motion commands while using LiDAR to improve environmental safety. This platform provides a more objective and scalable method for studying goldfish movement patterns, training response, and spatial learning over time.

I. Introduction

Goldfish navigation is useful for studying learning and memory because movement can be directly observed, measured, and compared across repeated trials. Goldfish can use visual cues, landmarks, and spatial relationships to move toward goals, which makes them a practical model for testing how training patterns affect behavior. However, many fish-learning studies rely heavily on manual observation. Manual tracking can miss small movements, introduce observer bias, and make it difficult to compare behavior across multiple fish or multiple training sessions.

This project addresses that problem by developing an autonomous fish-following robot that can track a goldfish in real time and respond to its movement. Instead of having researchers manually record where the fish swims, the system uses a camera, computer vision, LiDAR, and motor control to create a repeatable tracking platform. The robot detects fish position, calculates how far the fish is from the center of the frame, checks for obstacles using LiDAR, and sends movement commands to a motor controller. This creates a direct pipeline from fish behavior to robot motion.

The project builds on last year's fish-driven vehicle. Last year's system used a Raspberry Pi to process camera input and an Arduino Mega to control stepper motors. The final detection method used HSV color filtering through OpenCV because YOLO was too slow on the Raspberry Pi for real-time control. Although that system successfully tracked fish movement, it had several limitations. Motor control was time-based, which made movement less smooth and more dependent on vehicle weight and timing. LiDAR was proposed but not fully integrated. Mechanical problems such as chain slipping and jerky motion also limited the reliability of the system.

Go Fish 2.0 improves the system by changing the control approach and adding environmental sensing. This year's project found alternate methods to YOLO for detection robustness, keeps HSV detection for real-time speed, adds LiDAR-based obstacle detection, and replaces time-based motor commands with velocity-based motor commands. The novelty of the project is not just that the robot follows a fish, but that it creates a continuous closed-loop system where fish movement, obstacle detection, and motor response happen together in real time.

The biological goal is to use this system to study whether different training schedules affect learning and memory in goldfish. The project hypothesis is that goldfish trained with spaced repetition and breaks will learn faster and remember information longer than

fish trained without breaks. The technical goal is to create a stable platform that can collect movement data objectively enough to support that biological question.

II. Background

Goldfish have been used in studies of spatial learning because they can respond to visual cues and learn navigational tasks. Prior research has shown that fish can use landmarks and geometric information to orient themselves, which supports the idea that movement patterns can reflect learning. In goldfish and other teleost fish, the lateral pallium is considered functionally comparable to the mammalian hippocampus, a brain region involved in spatial memory and navigation. This provides the biological reasoning for using goldfish movement as a measurable indicator of learning.

The project also builds on previous fish-operated vehicle research. Givon et al. demonstrated that goldfish could learn to operate a vehicle and navigate toward visible targets outside of the tank. That study showed that fish can adapt to an unfamiliar relationship between swimming in water and movement on land. Last year's TJ project attempted to replicate and extend this idea using a fish-driven vehicle with a Raspberry Pi, Arduino Mega, camera, stepper motors, and a tank-mounted vehicle platform.

Last year's system first attempted to use YOLOv8 for fish detection. YOLO was attractive because it detects the fish as an object rather than only detecting color. However, when deployed on the Raspberry Pi with a webcam, YOLO was too slow for real-time motor response. The project therefore switched to HSV color filtering using OpenCV. HSV detection converted the frame into HSV color space, thresholded the fish's color range, located contours, filtered by area, and calculated the center of mass of the fish contour. This method ran at approximately 30 frames per second with minimal latency and became the final tracking method.

The current project uses that result as the starting point. HSV remains important because it is fast enough for real-time control on limited hardware. YOLO remains important because it is more robust to lighting changes, reflections, and complex backgrounds. This creates the central technical tradeoff of the project: HSV is faster, while YOLO is more accurate. Go Fish 2.0 tests both methods and uses them differently. YOLO is used as a high-accuracy detection baseline, while HSV is used for the real-time control loop when speed is more important. Your presentation specifically frames this as a speed-versus-accuracy tradeoff and describes the current approach as combining HSV-level speed with YOLO-level accuracy.

The project also adds LiDAR because last year's system lacked obstacle awareness. Without LiDAR, the robot could follow the fish into walls or maze boundaries, which could damage the vehicle, stress the fish, or distort behavioral data. The RPLIDAR A1M8 provides 360-degree distance measurements and allows the robot to stop or reduce forward motion when an obstacle is too close. This directly addresses a limitation from last year's system, where LiDAR was proposed but not implemented.

III. Applications

This project has applications in behavioral biology, robotics, and automated animal tracking. In biology, the system can be used to measure learning and memory in goldfish by recording movement patterns over repeated training trials. Instead of relying only on human observation, researchers can use position data, path consistency, time to target, and obstacle response to evaluate learning.

In robotics, the project demonstrates a real-time closed-loop control system. The robot uses sensor input, processes that input on a Raspberry Pi, applies control logic, and sends commands to a motor controller. This is a practical example of how computer vision and LiDAR can be combined for autonomous navigation.

In education, the project provides a low-cost platform for connecting biology, computer science, and engineering. Students can test different detection algorithms, compare control methods, improve mechanical design, and study animal behavior using one integrated system.

The system may also be adapted for other animal-tracking studies. With changes to the detection model and enclosure design, similar methods could be used to track other small aquatic animals or analyze movement patterns in controlled environments.

IV. Methods

System Architecture

The Go Fish 2.0 system contains five main subsystems: the camera, the Raspberry Pi, the LiDAR sensor, the VEX V5 Brain, and the robot chassis. The camera captures live video of the fish tank. The Raspberry Pi processes the video using HSV (Hue, Saturation, Value) color thresholding to detect the fish and determine its centroid position, calculates

movement values, reads LiDAR data, and generates movement commands. The LiDAR provides distance measurements used for obstacle detection and avoidance. The VEX V5 Brain receives movement commands from the Raspberry Pi and controls the drive motors. The robot chassis supports the electronics, sensors, and movement system.

The basic system architecture is:

Camera input → Raspberry Pi HSV vision processing → fish centroid position
LiDAR input → Raspberry Pi obstacle detection → safety override
Fish position + LiDAR data → velocity command generation
Velocity command → USB serial → VEX V5 Brain
VEX V5 Brain → motor control → robot movement

The system operates as a continuous feedback loop. During each cycle, the Raspberry Pi captures a camera frame, applies HSV color thresholding to isolate the fish from the background, identifies the fish contour, and calculates the fish's centroid position. The centroid offset from the center of the frame is then converted into velocity commands that determine the robot's movement direction. Simultaneously, LiDAR data is analyzed to determine whether obstacles are present in the robot's path. If an obstacle is detected, obstacle-avoidance behavior temporarily overrides the fish-following commands. The final velocity commands are transmitted to the VEX V5 Brain through USB serial connection, where the motors are updated accordingly. This process repeats continuously, allowing the robot to respond in real time to both fish movement and environmental obstacles.

Inputs and Outputs

The system has two main inputs. The first input is camera footage of the tank. The second input is LiDAR distance data from the surrounding environment. The camera input is processed using HSV color thresholding to determine the fish's position relative to the center of the frame. The LiDAR input is used to detect obstacles and determine whether avoidance behavior is necessary.

The output is a motor command sent from the Raspberry Pi to the VEX V5 Brain. Rather than controlling turning and steering, the command specifies horizontal and vertical movement velocities, also known as vx and vy. Vx represents left-right movement, and vy represents forward-backward movement.

For example, the command:

.5, 1

Indicates that the robot should move slightly to the right, then move forward. A positive or negative value of vx causes horizontal movement, while a positive or negative value of vy causes vertical movement. The VEX V5 Brain converts these velocity values into motor outputs for the robot's four wheel drive system. Two motors control movement along the forward-backward axis, while two motors control movement along the left-right axis. This configuration allows the robot to move directly up, down, left, or right without needing to rotate, enabling it to quickly reposition itself to follow the fish's movement within the tank.

Fish Detection Using HSV

HSV detection was used because it is fast enough to run on the Raspberry Pi in real time. Each camera frame is converted from RGB/BGR color space into HSV color space. A threshold is then applied to isolate the fish's color range. The resulting binary mask highlights pixels that likely belong to the fish. The program then identifies contours in the mask, filters out small contours, selects the largest valid contour, and calculates the centroid of that contour.

The centroid represents the fish's position:

fish position = (fx, fy)

The center of the camera frame is:

frame center = (cx, cy)

The system compares these values to determine how far the fish is from the center. This offset is then used to calculate movement commands.

HSV was selected for the real-time loop because it does not require neural network inference, does not load model weights, and uses lower CPU power than YOLO. This is important because delayed detection causes delayed motor response. HSV is faster than other methods tested because it uses pixel-level thresholding and avoids neural network inference, allowing higher frame rates on limited hardware.

Converting Detection to Movement

The major control change from last year is the shift from time-based movement to velocity-based movement. Last year's system sent an angle to the Arduino, and the Arduino converted that angle into motor run times. This made the system dependent on timing, vehicle weight, and mechanical consistency.

This year's system calculates velocity values from the fish's position. The horizontal offset is:

dx = fx - cx

The vertical offset is:

dy = cy - fy

These offsets are normalized:

vx = dx / frame_width

vy = dy / frame_height

The value vx represents turning speed. The value vy represents forward or backward speed. A deadzone is applied near the center of the frame to prevent jitter when the fish is close to the center.

The VEX V5 Brain converts vx and vy into left and right motor speeds using differential drive logic:

left = clamp(vy + vx, -1.0, 1.0)

right = clamp(vy - vx, -1.0, 1.0)

This approach allows the robot to make smooth corrections every frame instead of moving for a fixed number of milliseconds. The presentation identifies this as an improvement because velocity-based control updates constantly, reduces overshoot, and allows real-time adjustment.

LiDAR Safety System

The LiDAR system uses an RPLIDAR A1M8 sensor. The sensor provides 2D distance measurements around the robot. These measurements are divided into directional zones such as front, left, and right. The front zone is used to prevent collisions.

The basic safety rule is:

if distance < 30 cm and vy > 0:
 vy = 0

This prevents forward movement when an obstacle is within 30 cm. If the obstacle is closer than 20 cm, the robot can trigger an emergency stop. This means fish-following behavior can be overridden when safety requires it.

LiDAR was added because a fish-following robot without obstacle detection may move into walls or maze edges. That could damage the robot and create unreliable behavioral data. In this system, vision determines intended movement, while LiDAR enforces safety.

Raspberry Pi to VEX V5 Brain Communication

The Raspberry Pi sends motor commands to the VEX V5 Brain through USB serial communication. During system integration, USB connectivity was a major debugging challenge. The team used tools such as lsusb, dmesg, and serial device checks (/dev/ttyUSB* and /dev/ttyACM*) to identify connected hardware and verify communication between the Raspberry Pi and external devices. The LiDAR sensor typically appeared as a USB serial device, while the VEX V5 Brain required a reliable USB data cable connection to be recognized by the Raspberry Pi, as we were sending information regarding movement through the Pi.

A key lesson learned during development was that USB power and USB data transmission are not equivalent. Some cables powered the VEX V5 Brain LED but did not allow data communication. This meant the VEX V5 Brain appeared powered but did not show up as a USB serial device on the Raspberry Pi. The final system requires a stable data-capable USB connection so the Raspberry Pi can send continuous movement commands to the VEX V5 Brain.

Pseudocode

```
Initialize camera
Initialize LiDAR
```

```
Initialize serial connection to VEX V5 Brain
 Set frame center
 Set HSV threshold range
 Set obstacle thresholds

while robot is running:
 capture camera frame
 convert frame to HSV
 threshold fish color range
 find contours
 select largest valid contour
 calculate fish centroid
 compute dx and dy from frame center
 normalize dx and dy into vx and vy
 apply deadzone to reduce jitter
 read LiDAR scan
 calculate front distance
 if obstacle is too close:
 reduce or stop forward velocity
 convert vx and vy into left and right motor speeds
 send left and right speed values to VEX V5 Brain
 repeat
```

Metrics

The technical metrics for the system include detection frame rate, detection stability, serial command success rate, LiDAR scan reliability, motor response time, and obstacle avoidance accuracy. The biological metrics include time to target, path consistency, number of corrections, movement smoothness, and retention across training sessions.

What Was Tried and Did Not Work

YOLO was tested because it was expected to be more accurate than HSV. However, YOLO was too computationally expensive for real-time use on the Raspberry Pi, especially when motor response needed to update continuously. HSV was faster and more practical for the real-time loop.

The project also encountered USB communication issues. Some USB cables powered the VEX V5 Brain but did not transmit data. Battery and boost converter power setups were also tested but were unstable for early communication testing. Because the team was not connecting motors yet, the most reliable testing setup was to power and communicate with the VEX V5 Brain through USB while separately ensuring the Raspberry Pi had stable power.

V. Results

The completed Go Fish 2.0 system established a working design for an autonomous fish-following robot that connects fish detection, LiDAR obstacle sensing, and motor command generation in one control pipeline. The system successfully defined a real-time architecture in which the Raspberry Pi processes sensor inputs and sends movement commands to the VEX V5 Brain motor controller.

The project confirmed that HSV detection is the most practical method for real-time fish tracking on the Raspberry Pi. HSV detection is computationally lightweight and can support faster motor response than YOLO. YOLO provided a stronger detection baseline and showed high performance on synthetic validation data, with an mAP@50 of 0.995, but its real-time deployment remains limited by Raspberry Pi processing constraints and the synthetic nature of the dataset. YOLO had a lower system accuracy of .915, but the speed advantages gained by HSV outweighed the higher accuracy given by YOLO model.

The LiDAR subsystem was integrated into the design as a safety override. The RPLIDAR A1M8 was detected by the Raspberry Pi as a serial device, and the planned control logic uses forward distance measurements to prevent collisions when obstacles are within the threshold range. This improves on last year's system, where LiDAR was proposed but not implemented.

The project also established the serial communication structure between the Raspberry Pi and VEX V5 Brain. The VEX V5 Brain was successfully programmed through the Arduino IDE, and the command format was standardized as comma-separated left and right motor speeds. A major technical result was identifying that several USB cables could provide power without data. This finding helped isolate communication failures as hardware connection issues rather than code errors.

The engineering portion of the project was completed as a functional system design and implementation pipeline. The remaining biological training data should be added after repeated fish trials are completed.

Figure I. HSV-Based Fish Tracking and Centroid Detection.

The image above shows the output of the HSV-based computer vision pipeline used for fish detection. HSV color thresholding isolates the fish from the background, allowing the system to identify the fish contour, calculate its centroid position, and determine its displacement from the center of the camera frame. The displayed values include the centroid coordinates, horizontal and vertical offsets (dx, dy), and normalized velocity values (vx, vy) used to generate robot movement commands. The right image shows the corresponding original camera frame. This processing enables real-time tracking of fish movement and serves as the primary input for robot navigation.

```
Sent: CMD,-35,-35,18.4,-1,-1,0
LiDAR | Front: 18.3 cm | Left: 112.8 cm | Right: 22.7 cm
Sent: CMD,-35,-35,18.3,-1,-1,0
LiDAR | Front: 18.3 cm | Left: 112.8 cm | Right: 22.7 cm
Sent: CMD,-35,-35,18.3,-1,-1,0
LiDAR | Front: 18.4 cm | Left: 112.8 cm | Right: 22.6 cm
Sent: CMD,-35,-35,18.4,-1,-1,0
LiDAR | Front: 18.4 cm | Left: 112.8 cm | Right: 22.7 cm
Sent: CMD,-35,-35,18.4,-1,-1,0
LiDAR | Front: 18.4 cm | Left: 109.5 cm | Right: 22.6 cm
Sent: CMD,-35,-35,18.4,-1,-1,0
LiDAR | Front: 18.4 cm | Left: 112.4 cm | Right: 22.7 cm
Sent: CMD,-35,-35,18.4,-1,-1,0
```

Figure II. LiDAR Distance Measurements and Command Generation.

This figure shows real-time LiDAR data collected during robot operation. The system continuously measures distances in the forward, left, and right directions while simultaneously generating movement commands for the VEX V5 Brain. The displayed output includes motor command values and corresponding obstacle distances, allowing the robot to determine whether a clear path is available. When an obstacle is detected within a predefined threshol, the LiDAR subsystem can override fish-following behavior and trigger obstacle avoidance. These measurements provide environmental awareness and improve navigation safety during autonomous operation.

VI. Limitations

One limitation is that HSV detection is sensitive to lighting, reflections, and background colors. If the fish color overlaps with the environment or the lighting changes, the HSV mask may fail or detect noise. YOLO is more robust to these conditions, but it is too slow for real-time control on the Raspberry Pi without optimization.

Another limitation is that the YOLO dataset used in this stage was synthetic. The model performed well on synthetic validation data, but that does not guarantee strong performance on real tank footage. A larger dataset of real fish images from the actual tank environment would be needed for a stronger model.

A third limitation is hardware reliability. The project required stable USB communication among the Raspberry Pi, LiDAR, and VEX V5 Brain. Debugging showed that cables, hubs, and power limitations can prevent serial devices from being detected. This matters because the robot depends on continuous communication between the Raspberry Pi and VEX V5 Brain.

The mechanical design also limits performance. The robot must support a tank, electronics, and movement hardware while remaining stable and responsive. Last year’s project experienced chain slipping, noise, and jerky movement. Although this year’s velocity-based control improves the software side, the mechanical system still needs to be stable enough for smooth motion.

The biological experiment is also limited by sample size and training time. Fish behavior can vary between individuals, and stress from handling or changes in environment may

affect movement patterns. More trials are needed before drawing strong conclusions about spaced repetition and memory retention.

VII. Conclusion

Go Fish 2.0 completed an autonomous fish-following robot system that uses computer vision, LiDAR, Raspberry Pi processing, and VEX V5 Brain-based motor control. The project improves on last year's system by replacing time-based control with velocity-based control and adding LiDAR obstacle detection. HSV detection remains the best real-time method for Raspberry Pi control. The completed system creates a stronger platform for measuring goldfish behavior and studying how training schedules affect spatial learning over time.

VIII. Future Work and Recommendations

Future work should focus on collecting real tank footage to improve and evaluate YOLO under actual lighting conditions. The system should also add temporal smoothing, such as a Kalman filter, to reduce jitter in fish tracking and make motor response smoother. Finally, the completed robot should be used across repeated training sessions to compare learning and retention between the spaced-repetition group and the no-break training group.

IX. Materials

The project used a Raspberry Pi, DFRobot Romeo VEX V5 Brain-S3 motor controller, RPLIDAR A1M8 sensor, USB camera, VEX robotics kit components, fish tank, 12 in × 12 in × 12 in glass tank, black foam board, filter system, water testing kit, goldfish, goldfish pellets, USB data cables, and motor hardware. Software tools included Python, OpenCV, pySerial, Arduino IDE, YOLOv8, Google Colab, and LiDAR serial libraries.

References

Baratti, G., Potrich, D., Lee, S. A., Morandi-Raikova, A., & Sovrano, V. A. (2022). The geometric world of fishes: A synthesis on spatial reorientation in teleosts. *Animals, 12*(7), 881. https://doi.org/10.3390/ani12070881

Givon, S., Samina, M., Ben-Shahar, O., & Segev, R. (2022). From fish out of water to new insights on navigation mechanisms in animals. *Behavioural Brain Research, 419*. https://doi.org/10.1016/j.bbr.2021.113711

Saito, K., & Watanabe, S. (2005). Experimental analysis of spatial learning in goldfish. *The Psychological Record, 55*(4), 647–662. https://doi.org/10.1007/bf03395532

Ultralytics. (n.d.). *YOLOv8 documentation*. https://docs.ultralytics.com

OpenCV. (n.d.). *OpenCV documentation*. https://docs.opencv.org

DFRobot. (n.d.). *Romeo ESP32-S3 documentation*. https://wiki.dfrobot.com/SKU_DFR0975_Romeo_ESP32_S3

Slamtec. (n.d.). *RPLIDAR A1 documentation*. https://wiki.slamtec.com/display/SD/RPLIDAR+A1

Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3

Arduino. (n.d.). *Arduino IDE documentation*. https://docs.arduino.cc/software/ide

Link to GitHub: https://github.com/eshaMad81/Go-Fish-2.0-Project-2026